

Database Management Systems

CSE V-Semester

CS-502 Database Management Systems

Case Study of Relational Database Management Systems (RDBMS)

Overview

Relational Database Management Systems (RDBMS) are systems that use the relational model to manage and query structured data using SQL. Popular RDBMS examples include **Oracle**, **PostgreSQL**, and **MySQL**. This case study examines their architecture, features, and use cases.

Case Study: Oracle RDBMS

1. Overview

Oracle is a robust and enterprise-level RDBMS developed by Oracle Corporation, known for its advanced features and scalability.

- **Target Audience:** Large organizations and mission-critical applications.
- **Licensing:** Commercial, with some free versions (e.g., Oracle Express Edition).

2. Key Features

- **PL/SQL Support:** Combines SQL with procedural programming.
- **High Availability:** Supports clustering (RAC), replication, and fault tolerance.
- **Data Security:** Strong security features like data encryption and user roles.
- **Advanced Analytics:** Built-in tools for data mining and analytics.
- **Partitioning:** Optimizes the performance of large databases.

3. Example Use Case

A multinational banking institution uses Oracle to:

- Handle high-volume transactions with **ACID compliance**.
- Perform complex queries for real-time fraud detection.
- Implement disaster recovery using Oracle Data Guard.

Case Study: PostgreSQL

1. Overview

PostgreSQL is an open-source, object-relational database system known for extensibility and compliance with SQL standards.

- **Target Audience:** Developers and organizations needing flexibility.
- **Licensing:** Open-source under PostgreSQL License.

2. Key Features

- **JSON/JSONB Support:** Handles semi-structured data efficiently.

Database Management Systems

- **Advanced Indexing:** Includes GIN, GiST, and BRIN indexes.
- **Extensibility:** Supports user-defined functions, data types, and operators.
- **MVCC (Multi-Version Concurrency Control):** Provides high concurrency.
- **Foreign Data Wrappers (FDW):** Access external data sources like APIs and other databases.

3. Example Use Case

A startup builds an e-commerce platform using PostgreSQL to:

- Store structured customer and product data in relational tables.
- Manage inventory with **JSONB** for product metadata.
- Handle concurrent access by multiple users during peak sales.

Case Study: MySQL

1. Overview

MySQL, developed by Oracle Corporation, is a widely-used open-source RDBMS known for simplicity and speed.

- **Target Audience:** Web applications and small to medium-sized businesses.
- **Licensing:** Open-source under GPL, with commercial versions available.

2. Key Features

- **Simple Architecture:** Easy setup and management.
- **High Performance:** Optimized for read-heavy operations.
- **Replication:** Master-slave and group replication for scalability.
- **Storage Engines:** InnoDB for transactions, MyISAM for simplicity.
- **Cross-Platform Support:** Works on Windows, Linux, macOS.

3. Example Use Case

A blogging platform uses MySQL to:

- Store user-generated content in a relational structure.
- Ensure data consistency using InnoDB with ACID properties.
- Support global traffic using read replicas for scalability.

Comparison Table

Feature	Oracle	PostgreSQL	MySQL
License	Commercial	Open-source	Open-source + Commercial versions

Database Management Systems

Feature	Oracle	PostgreSQL	MySQL
Best For	Enterprise-level applications	Flexible and standards-compliant	Web apps and small businesses
Concurrency	Advanced locking mechanisms	MVCC	MVCC
Extensibility	Limited	High	Moderate
Data Types	Rich (LOBs, XML, JSON)	Extensive (JSONB, arrays)	Limited (JSON support via plugin)
Performance	High for complex queries	High for analytical workloads	High for simple workloads
Scalability	Excellent (RAC, Data Guard)	Good (sharding, replication)	Good (replication)

Summary of Insights

- **Oracle:** Ideal for large-scale, mission-critical applications requiring robust performance and advanced analytics.
- **PostgreSQL:** Suited for complex applications requiring customization and standards compliance.
- **MySQL:** Best for lightweight, read-heavy applications with simple requirements, like web apps.

Architecture, Physical Files, Memory Structures, and Background Processes in RDBMS

1. General RDBMS Architecture

An RDBMS architecture generally consists of:

- **Client Layer:** Interfaces like applications or tools for users to query and interact.
- **Server Layer:** The database engine that processes queries, manages transactions, and stores data.
- **Storage Layer:** Physical files where the data resides

Physical Files in RDBMS

RDBMS stores data in structured physical files on disk.

- **Data Files:** Store actual database data in tables.

Database Management Systems

- **Redo Logs:** Record changes for recovery in case of failure.
- **Control Files:** Contain metadata about the database structure.
- **Temporary Files:** Used for intermediate operations like sorting.
- **Index Files:** Maintain quick access paths for data retrieval.

Memory Structures

Memory is critical for query processing, caching, and performance optimization.

- **Buffer Cache:** Caches frequently accessed data blocks to reduce disk I/O.
- **Shared Pool:** Stores SQL statements, execution plans, and metadata for reuse.
- **Redo Log Buffer:** Temporarily stores redo log entries before writing to disk.
- **Sort Area:** Allocates memory for sorting operations.
- **Session Memory:** Isolates resources for individual user sessions.

Example (Oracle-Specific Memory Structure)

- **System Global Area (SGA):** Shared memory for all users.
- **Program Global Area (PGA):** Private memory for individual sessions.

Background Processes

Background processes handle essential operations like recovery, synchronization, and I/O.

Key Background Processes

1. **DB Writer (DBWn):** Writes dirty pages from buffer cache to disk.
2. **Log Writer (LGWR):** Writes redo log entries to disk from redo log buffer.
3. **Checkpoint (CKPT):** Signals DBWn to write all modified buffers to disk and updates control files.
4. **System Monitor (SMON):** Handles instance recovery after a crash.
5. **Process Monitor (PMON):** Cleans up processes after failures and releases resources.
6. **Archiver (ARCn):** Archives redo logs for backup and recovery.

MySQL Example

In MySQL:

- The **Query Optimizer** ensures efficient query execution.
- The **InnoDB Background Threads** include:
 - **Master Thread:** Manages I/O and transaction logs.
 - **Purge Thread:** Cleans up obsolete undo logs.

Example Use Case: Query Execution Process

1. **SQL Query Execution:**

Database Management Systems

- **Parsing:** The SQL statement is parsed in memory and checked for syntax.
- **Optimization:** Execution plans are generated and optimized using available indexes.
- **Execution:** Data is fetched or modified by accessing buffer caches or data files.

2. Write Operation:

- Changes are written to the **Redo Log Buffer** and marked as "dirty" in the **Buffer Cache**.
- Background processes (e.g., **DBWn**) write the dirty buffers to disk.

3. Recovery:

- During a failure, **SMON** and **LGWR** ensure data consistency by replaying redo logs and undoing incomplete transactions.

Summary Table

Component	Description
Physical Files	Data files, redo logs, control files, temporary files, index files.
Memory Structures	Buffer cache, shared pool, redo log buffer, sort area, session memory.
Background Processes	DBWn, LGWR, CKPT, SMON, PMON, ARCn.
Examples	Oracle (SGA/PGA), MySQL (InnoDB threads), PostgreSQL (WAL).

Data Dictionary and Dynamic Performance Views

Data Dictionary

The **data dictionary** is a set of read-only tables and views containing metadata about the database. It provides information about schema objects, users, storage structures, and system performance.

- **System-Owned Tables:** Automatically maintained by the database system.
- **Examples:**
 - USER_TABLES, ALL_TABLES, DBA_TABLES: Information about tables.
 - USER_INDEXES, ALL_INDEXES: Metadata about indexes.

Database Management Systems

- USER_TAB_COLUMNS: Column details of tables.

Dynamic Performance Views (V\$ Views)

Dynamic performance views monitor the real-time state of the database. These views are prefixed with **V\$**.

- **Examples:**
 - V\$SESSION: Current sessions in the database.
 - V\$SQL: Execution plans and query statistics.
 - V\$LOCK: Information on locks held by sessions.
 - V\$DATABASE: Details about the database instance.

Security, Role Management, and Privilege Management

Security

Security ensures only authorized users can access or manipulate data. Common mechanisms include:

- **Authentication:** Ensuring valid user credentials.
- **Encryption:** Encrypting data at rest and in transit.
- **Auditing:** Monitoring and logging user activities.

Role Management

Roles are collections of privileges that simplify permission management.

- **Predefined Roles:** Provided by the database (e.g., DBA, CONNECT).
- **Custom Roles:** Defined by administrators for specific use cases.
- **Granting and Revoking Roles:** Using SQL statements:

sql

```
CREATE ROLE analyst;
```

```
GRANT SELECT ON employees TO analyst;
```

```
GRANT analyst TO user1;
```

Privilege Management

Privileges are permissions for performing specific actions.

- **System Privileges:** Database-wide operations (e.g., CREATE TABLE).
- **Object Privileges:** Specific operations on objects (e.g., SELECT, INSERT).
- **SQL Statements:**

Sql

Database Management Systems

```
GRANT SELECT ON employees TO user1;
REVOKE INSERT ON employees FROM user1;
```

Profiles

Profiles define resource limits for users, such as:

- Password complexity.
- Session timeouts.
- SQL Example:
sql

```
CREATE PROFILE app_user LIMIT PASSWORD_LIFE_TIME 30;
ALTER USER user1 PROFILE app_user;
```

Invoker-Defined Security Model

- Determines whether the privileges of the invoker (user executing the function/procedure) or the definer (creator of the function/procedure) are used.
- **SQL Example:**
sql

```
CREATE OR REPLACE FUNCTION get_salary RETURN NUMBER AUTHID
CURRENT_USER IS
BEGIN
    -- Code accessing tables based on invoker's privileges.
END;
```

SQL Queries

Hierarchical Queries

Used to retrieve hierarchical data (e.g., organizational structures).

- SQL Example:
sql

Database Management Systems

```
SELECT employee_id, manager_id, LEVEL
FROM employees
CONNECT BY PRIOR employee_id = manager_id
START WITH manager_id IS NULL;
```

Inline Queries

A subquery defined within the main query, typically in the SELECT or FROM clause.

- SQL Example:

sql

```
SELECT emp_name, (SELECT dept_name FROM departments WHERE dept_id =
employees.dept_id)
FROM employees;
```

Flashback Queries

Recover past states of data using the database's undo logs.

- SQL Example:

sql

```
SELECT * FROM employees AS OF TIMESTAMP SYSDATE - INTERVAL '1'
DAY;
```

Summary Table

Concept	Description
Data Dictionary	Metadata tables (e.g., USER_TABLES, DBA_USERS)
Dynamic Views	Real-time database stats (V\$SESSION, V\$LOCK)
Security	Authentication, encryption, auditing
Role Management	Roles for grouping privileges (DBA, CONNECT)
Privilege Management	Grants/revokes object and system privileges
Profiles	Define user-specific resource constraints
Invoker Security Model	Uses invoker's privileges in procedures/functions

Database Management Systems

Concept	Description
SQL Types	Hierarchical queries, inline queries, flashback queries

Introduction to ANSI SQL

What is ANSI SQL?

ANSI SQL (American National Standards Institute SQL) is a standardized language for managing and manipulating relational databases. It defines a set of rules and syntax that ensure consistency across database systems.

Key Features of ANSI SQL

1. **Portability:** Queries written in ANSI SQL work on most RDBMS platforms (e.g., Oracle, MySQL, SQL Server).
2. **Standardized Operations:** Includes operations for data manipulation (CRUD), schema definition, and data control.
3. **Levels of Compliance:**
 - **Entry Level:** Basic features like SELECT, INSERT, UPDATE, DELETE.
 - **Intermediate Level:** Advanced features like joins, subqueries, and views.
 - **Full Level:** More sophisticated features like triggers and advanced data types.

ANSI SQL Components

1. **DDL (Data Definition Language):**
 - Used for schema-related operations.
 - Commands: CREATE, ALTER, DROP.

sql

```
CREATE TABLE employees (id INT, name VARCHAR(50), salary DECIMAL(10, 2));
```

2. **DML (Data Manipulation Language):**
 - Used for managing data.
 - Commands: SELECT, INSERT, UPDATE, DELETE.

sql

```
INSERT INTO employees (id, name, salary) VALUES (1, 'John Doe', 5000);
```

3. **DCL (Data Control Language):**

Database Management Systems

- Controls access to data.
- Commands: GRANT, REVOKE.

sql

GRANT SELECT ON employees TO user1;

4. **TCL (Transaction Control Language):**

- Manages transactions.
- Commands: COMMIT, ROLLBACK, SAVEPOINT.

sql

BEGIN;

UPDATE employees SET salary = salary + 500 WHERE id = 1;

COMMIT;

5. **SQL Extensions:** While ANSI SQL provides the foundation, most RDBMS systems extend it with proprietary features.

Cursor Management

What is a Cursor?

A cursor is a database object used to retrieve, manipulate, and navigate through rows in a query result set sequentially.

Types of Cursors

1. **Implicit Cursors:** Automatically created by the database for single-row queries.
2. **Explicit Cursors:** Defined by users to handle multi-row queries.

Cursor Management in SQL

Basic Cursor Lifecycle:

1. **Declare:** Define the cursor and its associated query.
2. **Open:** Initialize and populate the cursor with query results.
3. **Fetch:** Retrieve one row at a time from the cursor.
4. **Close:** Release the cursor and its resources.

Database Management Systems

Nested Cursors

Nested cursors are cursors defined and used inside another cursor. They allow processing complex multi-level query results.

- **Example:**

Sql

```
DECLARE outer_cursor CURSOR FOR SELECT department_id FROM departments;  
DECLARE inner_cursor CURSOR FOR SELECT employee_id FROM employees WHERE  
department_id = ?;
```

```
OPEN outer_cursor;  
FETCH NEXT FROM outer_cursor INTO @dept_id;
```

```
WHILE @@FETCH_STATUS = 0  
BEGIN
```

```
    OPEN inner_cursor FOR SELECT employee_id FROM employees WHERE  
    department_id = @dept_id;  
    FETCH NEXT FROM inner_cursor INTO @emp_id;
```

```
    WHILE @@FETCH_STATUS = 0  
    BEGIN
```

```
        PRINT @emp_id; -- Process the employee ID  
        FETCH NEXT FROM inner_cursor INTO @emp_id;  
    END;
```

```
    CLOSE inner_cursor;  
    FETCH NEXT FROM outer_cursor INTO @dept_id;  
END;
```

```
CLOSE outer_cursor;
```

Database Management Systems

Parameterized Cursors

Parameterized cursors accept input parameters to customize query execution.

- **Example:**

sql

```
DECLARE employee_cursor CURSOR FOR SELECT employee_id, name FROM  
employees WHERE department_id = @dept_id;
```

```
SET @dept_id = 10; -- Pass parameter value
```

```
OPEN employee_cursor;
```

```
FETCH NEXT FROM employee_cursor INTO @emp_id, @emp_name;
```

```
WHILE @FETCH_STATUS = 0
```

```
BEGIN
```

```
    PRINT @emp_id + ' ' + @emp_name;
```

```
    FETCH NEXT FROM employee_cursor INTO @emp_id, @emp_name;
```

```
END;
```

```
CLOSE employee_cursor;
```

Key Points

1. **ANSI SQL** ensures cross-platform compatibility and standardization.
2. **Cursors** provide controlled access to query results, especially useful in procedural logic.
3. **Nested and Parameterized Cursors** handle complex scenarios like multi-level queries and dynamic data retrieval.

Database Management Systems

Stored Procedures

What is a Stored Procedure?

A **stored procedure** is a precompiled collection of one or more SQL statements that can be executed as a single unit. Stored procedures are stored within the database and are executed by calling their name. They help improve performance by reducing the need to send multiple SQL queries from the client application.

Usage of Parameters in Stored Procedures

Parameters in stored procedures allow dynamic and flexible execution. Stored procedures can accept **input** parameters, **output** parameters, or **input/output** parameters.

1. **Input Parameters:** Used to pass values into the stored procedure.

sql

```
CREATE PROCEDURE GetEmployeeDetails (IN emp_id INT)
BEGIN
    SELECT name, salary FROM employees WHERE id = emp_id;
END;
```

2. **Output Parameters:** Used to return values from the procedure to the caller.

sql

```
CREATE PROCEDURE GetEmployeeSalary (IN emp_id INT, OUT emp_salary
DECIMAL)
BEGIN
    SELECT salary INTO emp_salary FROM employees WHERE id = emp_id;
END;
```

3. **Input/Output Parameters:** Can both receive and return values.

sql

```
CREATE PROCEDURE UpdateSalary (INOUT emp_id INT, IN salary_change
DECIMAL)
BEGIN
    UPDATE employees SET salary = salary + salary_change WHERE id = emp_id;
    SELECT salary FROM employees WHERE id = emp_id;
```

Database Management Systems

END;

User-Defined Functions (UDFs)

What is a User-Defined Function?

A **User-Defined Function (UDF)** is a function created by the user in the database that can be used to perform specific operations and return a result. UDFs can be scalar (returning a single value) or table-valued (returning a table).

- **Scalar UDF:** Returns a single value.

sql

```
CREATE FUNCTION GetEmployeeSalary (emp_id INT)
RETURNS DECIMAL
BEGIN
    DECLARE emp_salary DECIMAL;
    SELECT salary INTO emp_salary FROM employees WHERE id = emp_id;
    RETURN emp_salary;
END;
```

- **Table-Valued UDF:** Returns a table of data.

sql

```
CREATE FUNCTION GetEmployeesByDept (dept_id INT)
RETURNS TABLE
AS
RETURN (SELECT name, salary FROM employees WHERE department_id =
dept_id);
```

Limitations of User-Defined Functions

- **Side Effects:** UDFs should be free from side effects, such as modifying the database state (i.e., no INSERT, UPDATE, or DELETE operations).
- **Performance:** They can be less efficient in certain cases, especially when used in large result sets.
- **Limited Execution Context:** UDFs may not have access to certain system-level resources, and their execution is generally restricted to return a value without side effects.

Database Management Systems

Triggers

What is a Trigger?

A **trigger** is a stored procedure that automatically executes in response to certain events on a table or view, such as INSERT, UPDATE, or DELETE. Triggers are typically used for enforcing business rules, auditing, and preventing invalid transactions.

- **Example of a Trigger:**

sql

```
CREATE TRIGGER BeforeEmployeeInsert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be
negative';
    END IF;
END;
```

Mutating Table Errors in Triggers

A **mutating table error** occurs when a trigger tries to modify or query the same table that caused the trigger to fire. This leads to inconsistency or recursion.

- **Example:** If an AFTER UPDATE trigger tries to query or update the same table (e.g., employees), it causes a mutating table error because the data in the table is in an inconsistent state.

sql

```
CREATE TRIGGER AfterEmployeeUpdate
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    UPDATE employees SET salary = salary * 1.1 WHERE department_id =
:NEW.department_id;
END;
```

Database Management Systems

-- This may cause a mutating error because we are modifying the same table within the trigger.

Instead of Triggers

An **INSTEAD OF trigger** allows you to define a custom action for a DML operation (INSERT, UPDATE, or DELETE) instead of performing the default operation. This is useful for views or complex logic that cannot directly be handled with the base table.

- **Example of an INSTEAD OF Trigger:**

sql

```
CREATE TRIGGER InsteadOfInsertForView
```

```
INSTEAD OF INSERT ON employee_view
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO employees (id, name, salary) VALUES (:NEW.id, :NEW.name,  
:NEW.salary);
```

```
END;
```

- In this example, the INSTEAD OF trigger ensures that when an insert is attempted on the employee_view, the trigger inserts the data into the actual employees table.

Key Points Summary

Concept	Description
Stored Procedures	Precompiled SQL code executed as a unit; can accept input, output, or I/O parameters.
User-Defined Functions	Functions created by users to perform calculations or return values. Limitations include no side effects (no INSERT, UPDATE, or DELETE).
Triggers	Automated procedures that execute on data changes. Useful for auditing or enforcing rules.
Mutating Errors	Occurs when a trigger modifies or queries the same table that triggered it.
INSTEAD OF Triggers	Customizes the actions for DML operations on views or tables. Executes instead of the default action.